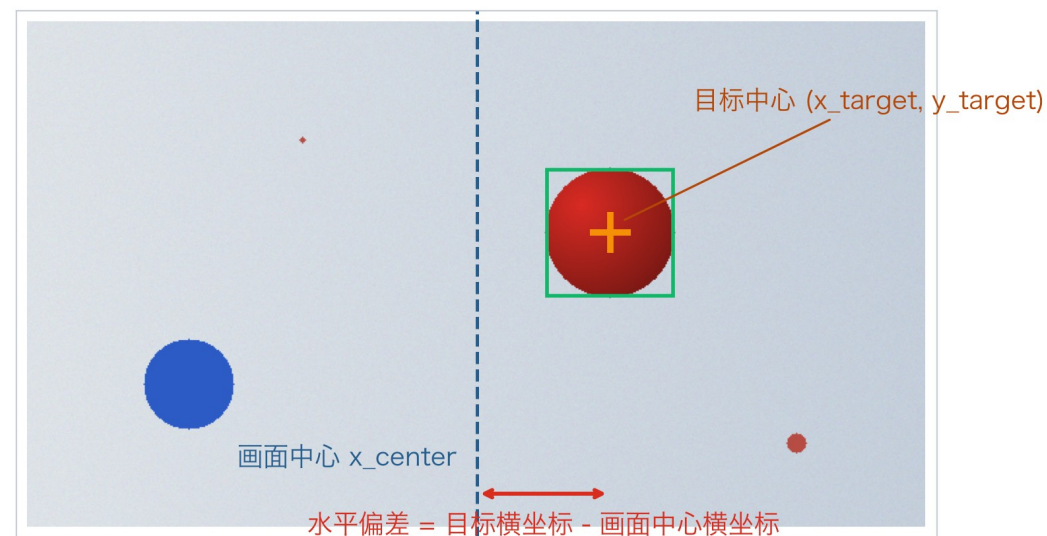


机器人视觉第一课

让 AnyCar 「看见」 彩色目标

传统视觉与 OpenCV 初步 | 预计授课 10 分钟

浙江大学生机器人协会 · 教学部内训选题



感知、决策与执行



对初学者来说，难点几乎都集中在中间那一步：画面里的东西很多，程序怎么知道哪一堆像素才是目标。这节课把这一步拆成四个可以单独检查的小步骤。

处理一帧画面的六个步骤



每一步都留下可以直接看到的中间结果：掩膜、去噪后的掩膜、轮廓、中心点。程序出错时，先看是哪一步的产物不对，再回头改那一步。

本节课重点演示前三步到第五步，第六步只做到能打印出转向量。

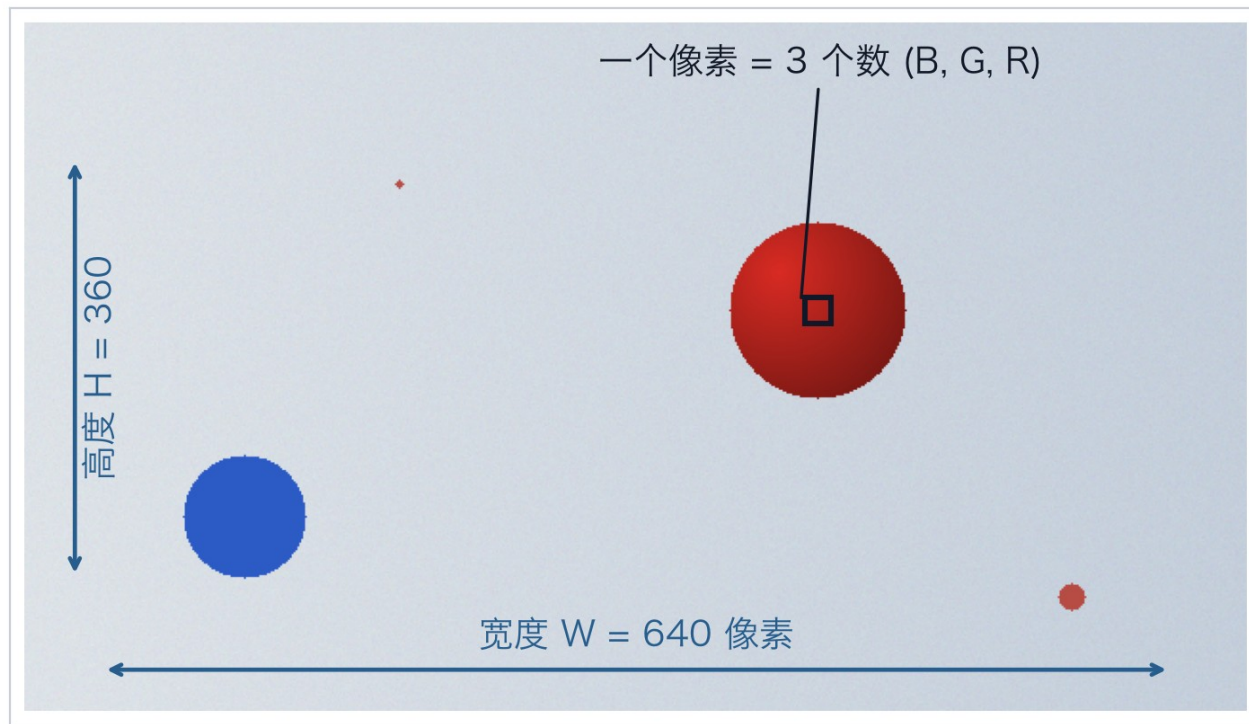
一帧画面在内存里就是一个矩阵

frame 是「高 $\times$ 宽 $\times$ 3」的数组

三个通道分别存放蓝、绿、红的亮度

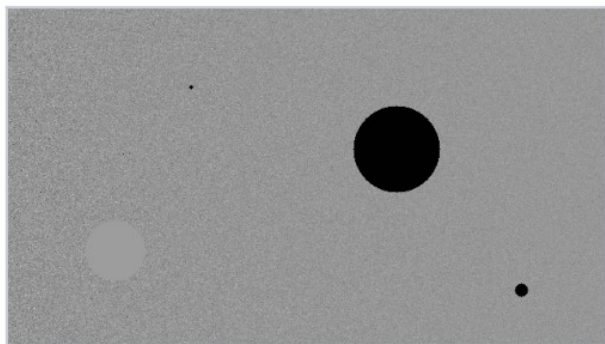
所谓图像处理，就是对这些数字做运算

数组的形状决定了后面所有操作：行是高度，列是宽度，第三个维度才是颜色。取一个像素就是取三个数字。

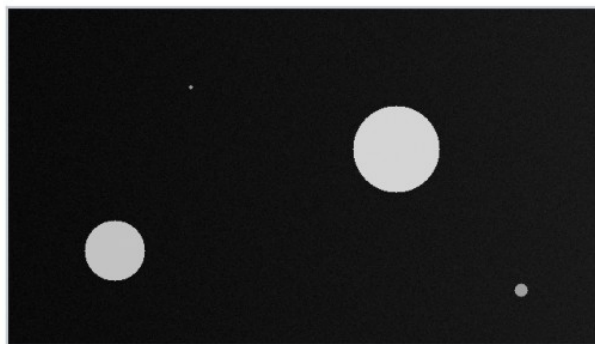


HSV 把颜色和亮度分开

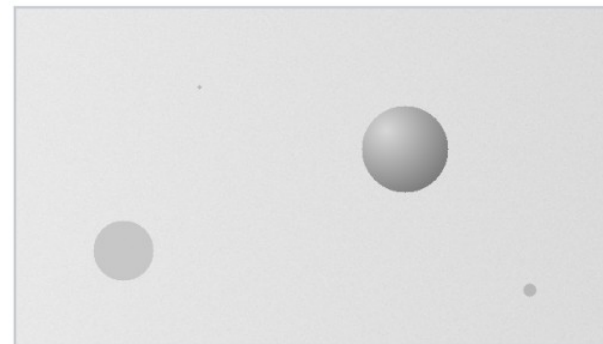
H 色调 (是什么颜色)



S 饱和度 (颜色有多纯)



V 明度 (有多亮)



H 表示是什么颜色， S 表示颜色有多纯， V 表示有多亮

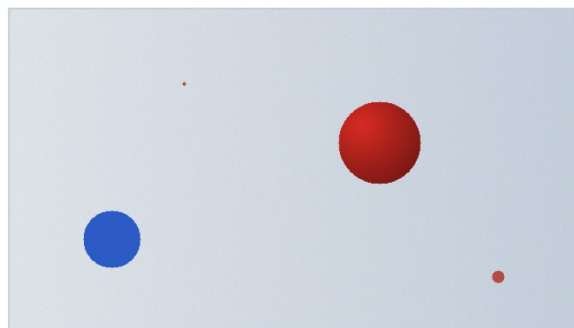
光照变强时， RGB 的三个通道一起变， 阈值很容易失效； HSV 里亮度主要影响 V， 颜色范围更稳

颜色阈值：把想要的像素留下来

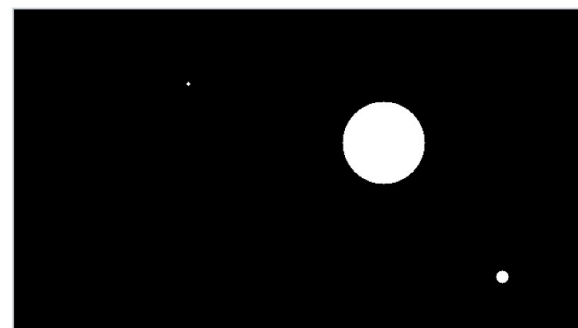
```
hsv = cv2.cvtColor(frame,  
                    cv2.COLOR_BGR2HSV)  
  
lower = np.array([ 0, 120, 90])  
upper = np.array([ 10, 255, 255])  
  
mask = cv2.inRange(hsv, lower, upper)
```

范围内的像素写成 255，范围外写成 0。得到的二值图就是掩膜，白色保留目标，黑色丢掉。

① 摄像头拍到的一帧



② 红色阈值分割后的掩膜



红色在 HSV 色环的两端，所以 H 的上界和下界要各取一段，两段结果再合并。

常见颜色的 HSV 起点

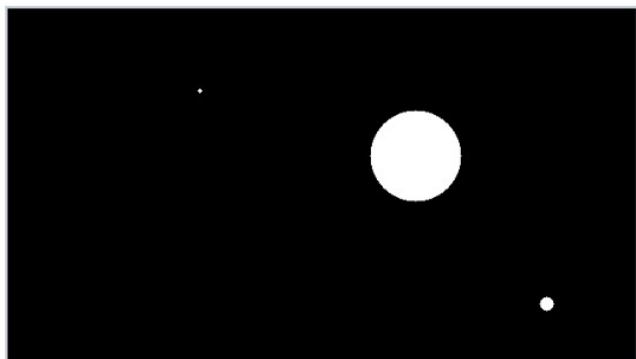
颜色	H（色调）	S（饱和度）	V（明度）
红	0-10 或 170-179	120-255	90-255
绿	40-80	80-255	60-255
蓝	100-130	120-255	70-255

OpenCV 的 H 取值是 0 到 179，不是常见的 0 到 360，写阈值前先想清楚这一点

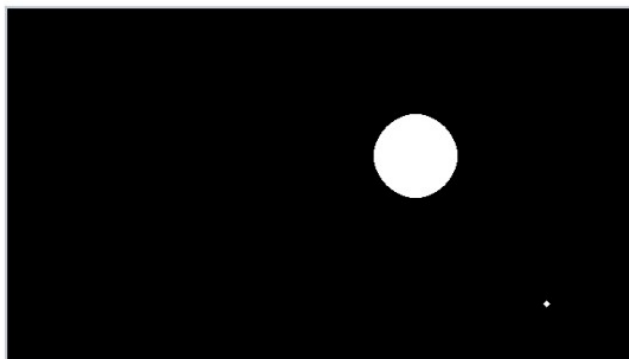
这张表只是起点。同一块红色在灯光下和在阴影里，V 的下界可能差很多，必须对着现场微调

形态学：先腐蚀，再膨胀

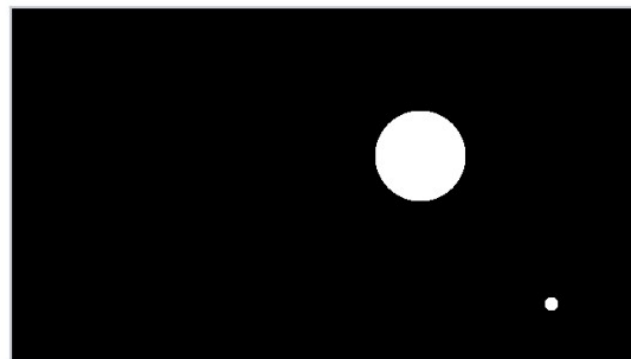
原掩膜：目标 + 两处杂点



腐蚀：小杂点消失



再膨胀：目标恢复



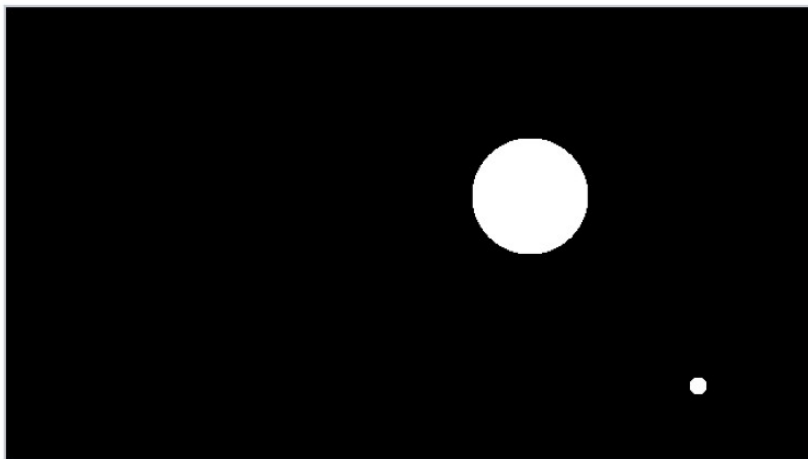
```
kernel = np.ones((7, 7), np.uint8)
mask = cv2.morphologyEx(mask,
                        cv2.MORPH_OPEN, kernel)
```

腐蚀让目标缩一圈，比核还小的杂点直接消失

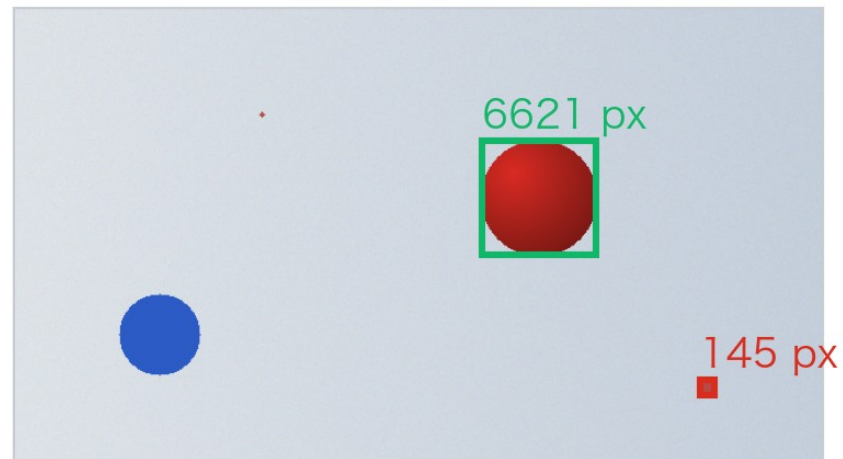
膨胀再让目标长回来，但杂点不会复活

轮廓与面积筛选

去噪后的掩膜



面积筛选：只保留大目标



```
cnts, _ = cv2.findContours(mask,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE)
```

findContours 把白色区域连成一圈一圈的边界
再用 contourArea 按面积筛掉太小的轮廓

求目标中心

$$\bar{x} = (1 / A) \cdot \sum u$$

$$\bar{y} = (1 / A) \cdot \sum v$$

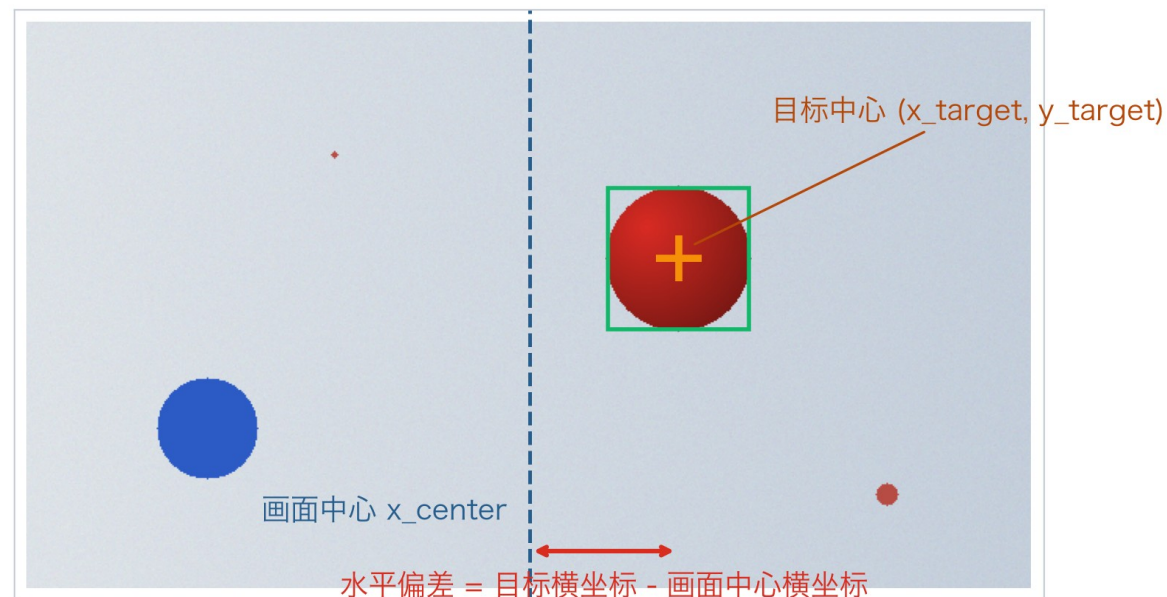
A 是区域面积, (u, v) 是区域内每个像素的坐标

```
M = cv2.moments(c)
```

```
cx = int(M['m10'] / M['m00'])
```

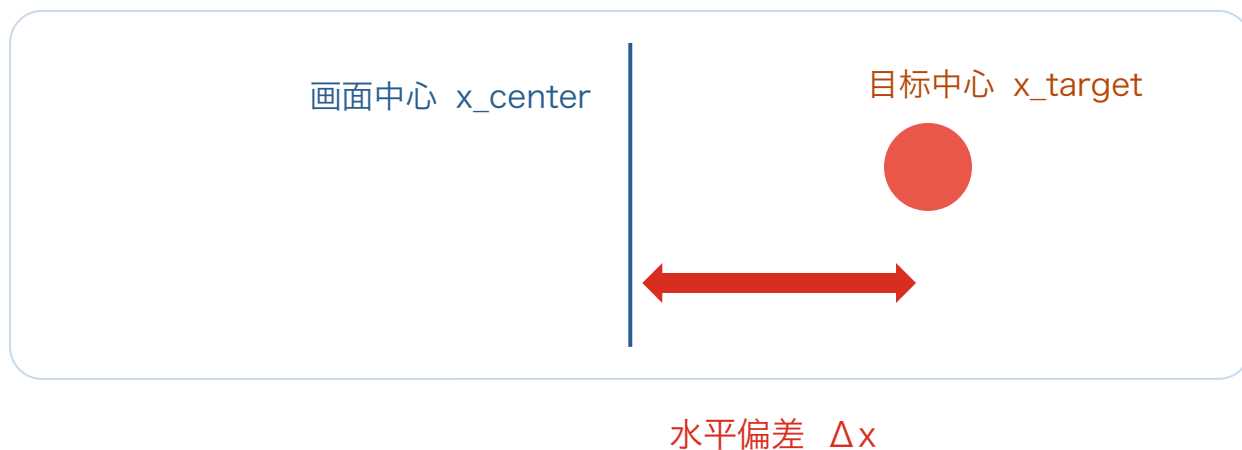
```
cy = int(M['m11'] / M['m00'])
```

图像矩就是把上面两个求和写成紧凑形式。这一帧里目标面积 6621 像素, 算出的中心是 (415, 150)。



把位置偏差变成转向量

$$\omega = k_p \cdot (x_{\text{target}} - x_{\text{center}})$$



Δx 为正，目标在右侧，就向右转

Δx 越大，转得越快，比例由 k_p 决定

k_p 调太大，车会在目标两侧来回摆

这一步不需要完整的控制算法。只要能把偏差换算成一个带正负号的转向量，视觉和运动之间就接上了。

最小可运行脚本

```
import cv2, numpy as np

cap = cv2.VideoCapture(0)
while True:
    ok, frame = cap.read()
    if not ok:
        break

    hsv = cv2.cvtColor(frame,
                        cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv,
                       np.array([0, 120, 90]),
                       np.array([10, 255, 255]))
    mask = cv2.morphologyEx(mask,
                           cv2.MORPH_OPEN,
                           np.ones((7, 7), np.uint8))
```

```
cnts, _ = cv2.findContours(mask,
                           cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)
if cnts:
    M = cv2.moments(
        max(cnts, key=cv2.contourArea))
    cx = int(M['m10'] / M['m00'])
    err = cx - frame.shape[1] // 2
    omega = 0.005 * err      # k_p
    print(f'err={err:4d}px'
          f' {omega:+.3f} rad/s')

    cv2.imshow('mask', mask)
    if cv2.waitKey(1) == 27: # ESC
        break
cap.release(); cv2.destroyAllWindows()
```

视觉最容易失效的四种情况

现象	可能的原因	先试的办法
掩膜突然变空	光线变了，原来的 H 或 V 范围失效	固定光源，或把 V 的下界调低再试
目标旁边多出一个框	掩膜里有颜色相近的干扰物	用面积筛选，或只在画面中部找目标
中心点反复跳动	目标被部分遮挡，轮廓形状在变	连续几帧取平均，丢帧时保持上次结果
目标贴到边缘就丢	目标跑出了视野范围	先原地转动找回目标，再做伺服

把这四条抄下来，自己动手时遇到新问题就往后加一行。这份清单比一段跑得通的代码更值钱。

小结与课下任务

一帧画面是高度 $\times$ 宽度 $\times$ 3 的数字矩阵

HSV 把颜色和亮度分开，比 RGB 更适合定阈值

先腐蚀后膨胀去掉杂点，再用轮廓和面积筛出目标

目标中心与画面中心的差，就是转向量的来源

课下任务

跑通最小脚本，保存一张分割成功的截图

记下三条失效现象，写清现象、原因和试过的办法

加分项：把转向量接到 AnyCar 或串口上真跑一次